Studio Nx

백엔드 277 — 클립 서비스

2026-10-19

백엔드 277 — 클립 서비스 (services/clip_service.py)

항목	내용
파일	`studionx/backend/app/services/clip_service.py`
작업	S1-10a 클립·연결·패턴 API: 타임라인 클립의 목록, 만들기, 수정, 삭제 서비스 (PRD 02 FR-2196)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전 · 다음	이전 276 클립·연결 스키마, 다음 278 연결 서비스

1. 하는 일

트랙에 놓인 클립 한 건을 읽고 쓰는 일을 모두 담당합니다. 권한과 404 규칙을 이 파일 한 곳에 모아 두어서, 라우터(280)는 요청을 그대로 넘기기만 하면 됩니다.

```
list_clips(db, user, track_id)          트랙 확인 → 클립 목록을 start_tick 순으로 반환
create_clip(db, user, track_id, data)   트랙 권한 확인 → MIDI 는 패턴, AUDIO 는 샘플 확인 → 삽입
update_clip(db, user, clip_id, data)    클립 찾기 → 트랙 권한 확인 → 보낸 칸만 바꾼 뒤 반환
delete_clip(db, user, clip_id)          클립 찾기 → 트랙 권한 확인 → 삭제하고 패턴은 남김
pattern_is_used(db, pattern_id)         clips 표에서 pattern_id 검색 → 한 건이라도 있으면 True
```

만들기 조건은 다음과 같습니다. MIDI 클립의 패턴은 같은 프로젝트의 것이어야 하며, 없거나 남의 프로젝트 것이면 404로 막습니다. AUDIO 클립의 샘플은 존재해야 하며, 없으면 404로 막습니다. 종류와 가리키는 칸이 어긋나는 요청은 요청 스키마(276)가 422로 먼저 막습니다. 조회는 멤버면 누구나 되고, 쓰기는 OWNER와 EDITOR만 할 수 있습니다.

2. 코드

```
def _require_editor_track(db: Session, user: User, track_id: int) -> Track:
    """쓰기용 트랙 확인 — 남의 트랙은 404, 권한이 없으면 403."""
    track = track_service.require_track(db, user, track_id)
    _project, member = project_service.get_membership(db, track.project_id, user)
    project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)
    return track


def _require_clip(db: Session, user: User, clip_id: int) -> Clip:
    """클립 한 건. 없거나 남의 것이면 404 (권한은 트랙 쪽에서 다시 본다)."""
    clip = db.get(Clip, clip_id)
    if clip is None:
        raise AppError(ErrorCode.NOT_FOUND, 404, "Clip not found.")
    track = track_service.require_track(db, user, clip.track_id)
    if track.track_id != clip.track_id:  # 방어: require_track 이 다른 트랙을 돌려줄 일은 없음
        raise AppError(ErrorCode.NOT_FOUND, 404, "Clip not found.")
    return clip
```

`create_clip` 은 만들기, `delete_clip` 은 삭제, `pattern_is_used` 은 패턴 삭제 전 확인을 맡습니다.

```
def create_clip(
    db: Session, user: User, track_id: int, data: ClipCreateRequest
) -> Clip:
    """트랙에 클립을 놓는다 (FR-2196). OWNER·EDITOR 만.

    MIDI 클립의 패턴은 같은 프로젝트의 것이어야 하고(404), AUDIO 클립의 샘플은
    있어야 한다(404). 종류와 가리키는 칸이 어긋나는 것은 요청 스키마가 422 로 막는다.
    """
    track = _require_editor_track(db, user, track_id)

    if data.clip_type == ClipType.MIDI:
        pattern = db.get(Pattern, data.pattern_id)
        if pattern is None or pattern.project_id != track.project_id:
            raise AppError(ErrorCode.NOT_FOUND, 404, "Pattern not found.")
    else:
        sample = db.get(Sample, data.sample_id)
        if sample is None:
            raise AppError(ErrorCode.NOT_FOUND, 404, "Sample not found.")

    clip = Clip(
        track_id=track.track_id,
        clip_type=data.clip_type,
        sample_id=data.sample_id,
        pattern_id=data.pattern_id,
        name=data.name,
        start_tick=data.start_tick,
        length_tick=data.length_tick,
        offset_tick=data.offset_tick,
        gain_db=data.gain_db,
        is_looped=data.is_looped,
        fade_in_tick=data.fade_in_tick,
        fade_out_tick=data.fade_out_tick,
        fade_in_curve=data.fade_in_curve,
        fade_out_curve=data.fade_out_curve,
        warp_mode=data.warp_mode,
        warp_markers=data.warp_markers,
        pitch_semitones=data.pitch_semitones,
        is_reversed=data.is_reversed,
    )
    db.add(clip)
    db.commit()
    db.refresh(clip)
    return clip


def delete_clip(db: Session, user: User, clip_id: int) -> None:
    """클립 삭제 (FR-2196). OWNER·EDITOR 만. 클립이 가리키던 패턴은 그대로 남는다."""
    clip = _require_clip(db, user, clip_id)
    _require_editor_track(db, user, clip.track_id)
    db.delete(clip)
    db.commit()


def pattern_is_used(db: Session, pattern_id: int) -> bool:
    """이 패턴을 가리키는 클립이 있는지 — 패턴 삭제 전에 본다 (FR-2196)."""
    return (
```

```
        db.scalar(select(Clip.clip_id).where(Clip.pattern_id == pattern_id).limit(1))
        is not None
    )
```

문법 풀이

식	뜻
`track_service.require_track(db, user, track_id)`	트랙이 없거나 남의 프로젝트 것이면 404로 막습니다
`project_service.require_role(member, MemberRole.OWNER, MemberRole.EDITOR)`	VIEWER가 쓰기를 시도하면 403으로 막습니다
`db.get(Pattern, data.pattern_id)`	패턴 한 건을 읽고, 없으면 404로 막습니다
`pattern.project_id != track.project_id`	남의 프로젝트 패턴을 가리키는 클립은 404로 막습니다
`db.add(clip)`	클립 한 행을 세션에 올립니다
`db.commit()`	지금까지의 변경을 DB에 한 번에 확정합니다
`db.refresh(clip)`	확정 직후 DB에 적힌 최신 값(번호, 시각)을 다시 읽습니다
`data.model_dump(exclude_unset=True)`	요청에 실제로 담긴 칸만 골라서 사전으로 만듭니다
`setattr(clip, field, value)`	골라낸 칸 이름으로 클립의 값을 바꿉니다. 보낸 칸만 바뀝니다
`select(Clip.clip_id).where(...).limit(1)`	행을 채우지 않게 번호 한 개만 찾아 있고 있는지 봅니다

3. 왜 이렇게

선택	이유
쓰기 권한을 `_require_editor_track` 한 곳에 모음	만들기, 수정, 삭제가 같은 규칙을 씁니다. 남의 트랙은 404, 권한 없음은 403입니다
클립 존재와 트랙 권한을 `_require_clip` 이 함께 확인	클립 번호로 먼저 찾은 뒤 그 클립의 트랙에 대해 권한을 다시 확인합니다
수정은 보낸 칸만 바꿈	PATCH 요청은 부분 수정이라 보내지 않은 칸은 그대로 둡니다
삭제가 패턴을 함께 지우지 않음	패턴은 여러 클립이 쓸 수 있고, 지우면 노트가 통째로 사라집니다
`pattern_is_used` 를 클립 쪽에 둠	패턴을 쓰는 클립이 있는지 묻는 질문이라 클립 표를 가장 잘 아는 이 파일이 답합니다
목록을 `start_tick` 순으로 정렬	타임라인을 왼쪽부터 그리는 화면의 순서와 같습니다

4. 검증

`tests/test_clip_routes.py` 열한 건이 모두 통과합니다. `test_create_midi_clip_201` 과 `test_create_audio_clip_201` 은 종류별 만들기 201을, `test_create_clip_to_other_project_pattern_is_404` 는 남의 프로젝트 패턴 404를, `test_create_clip_nonexistent_sample_is_404` 는 없는 샘플 404를 확인합니다. `test_patch_clip` 은 보낸 칸만 바뀌는지, `test_delete_clip` 은 204인지를, `test_viewer_can_read_but_not_write` 와 `test_stranger_gets_404` 와 `test_needs_login` 은 권한과 로그인 규칙을 확인합니다.

`tests/test_full_project_routes.py` 의 `test_full_includes_clips_and_connections` 는 전체 조회에 클립이 담기는지 봅니다. `pytest -q` 는 1083 passed, `ruff check .` 는 오류 없이 통과합니다.

5. 남은 일 (사용자)

확인할 것	내용
문서 검토	S1-10a 서비스와 라우터 문서(277 부터 281 까지)가 모두 끝난 뒤 한 번에 검토합니다
API 실제 호출 확인	화면에서 클립을 놓고 옮기고 지우는 동작을 시퀀서에서 직접 확인합니다